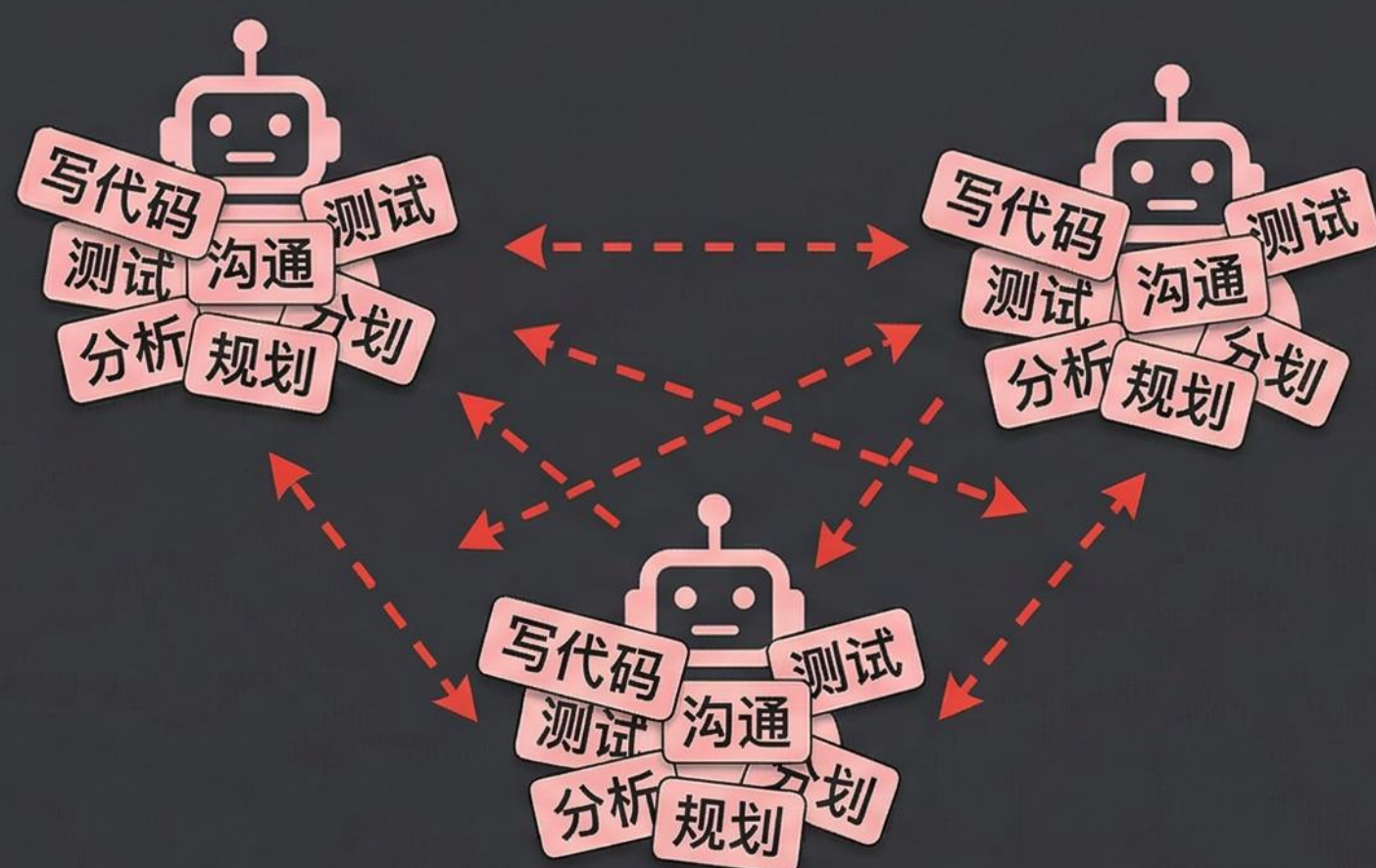


25 | 团队角色体系

分工设计与行为规范文件系统记忆

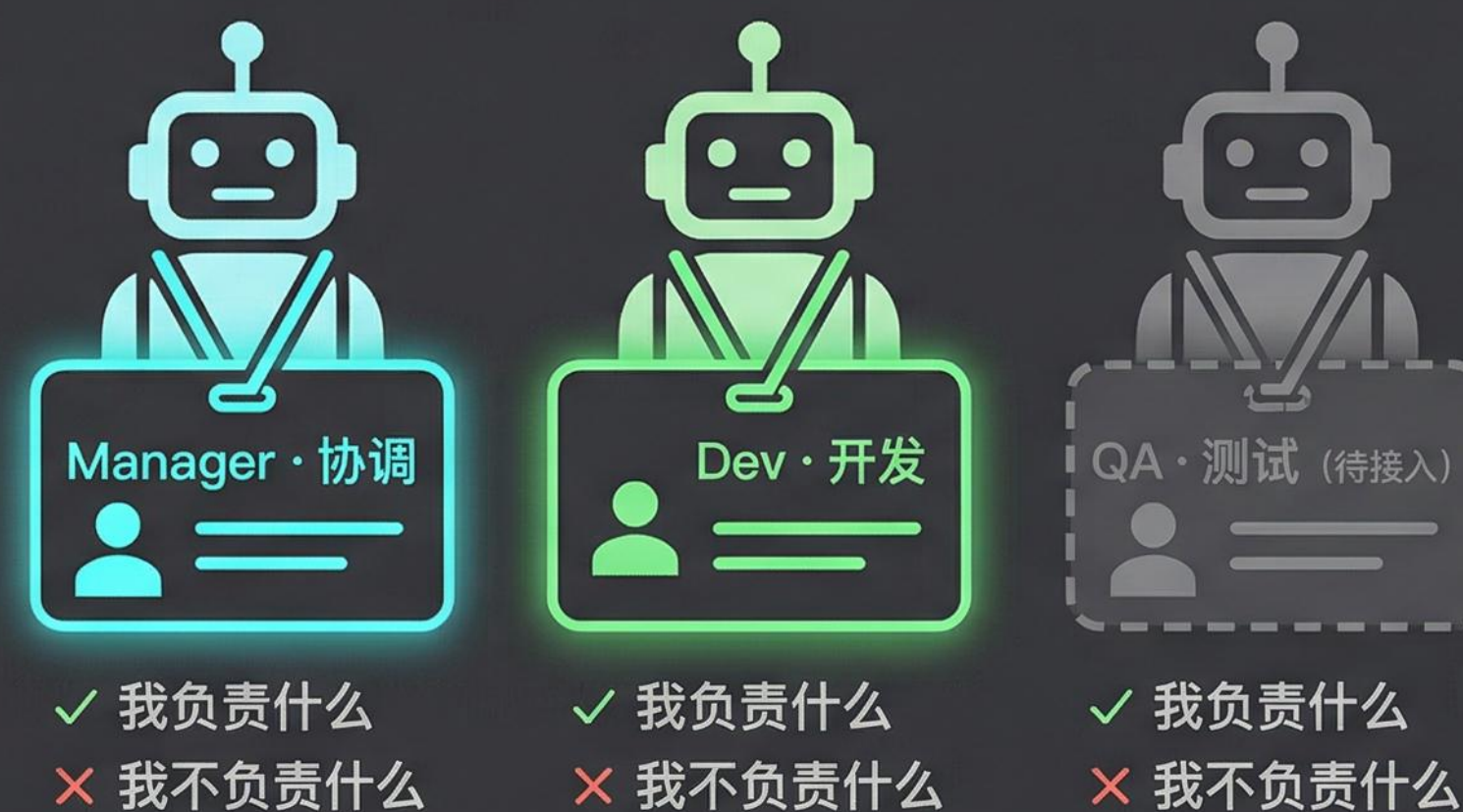
没有角色体系的多 Agent，不是团队，是一堆人

✗ 通才 Agent 堆



- ① 上下文塞满了什么都能做的知识
- ② 记忆往四面八方积累，方向分裂
- ③ 遇到选择题，判断方向随机

✓ 有设计的角色体系



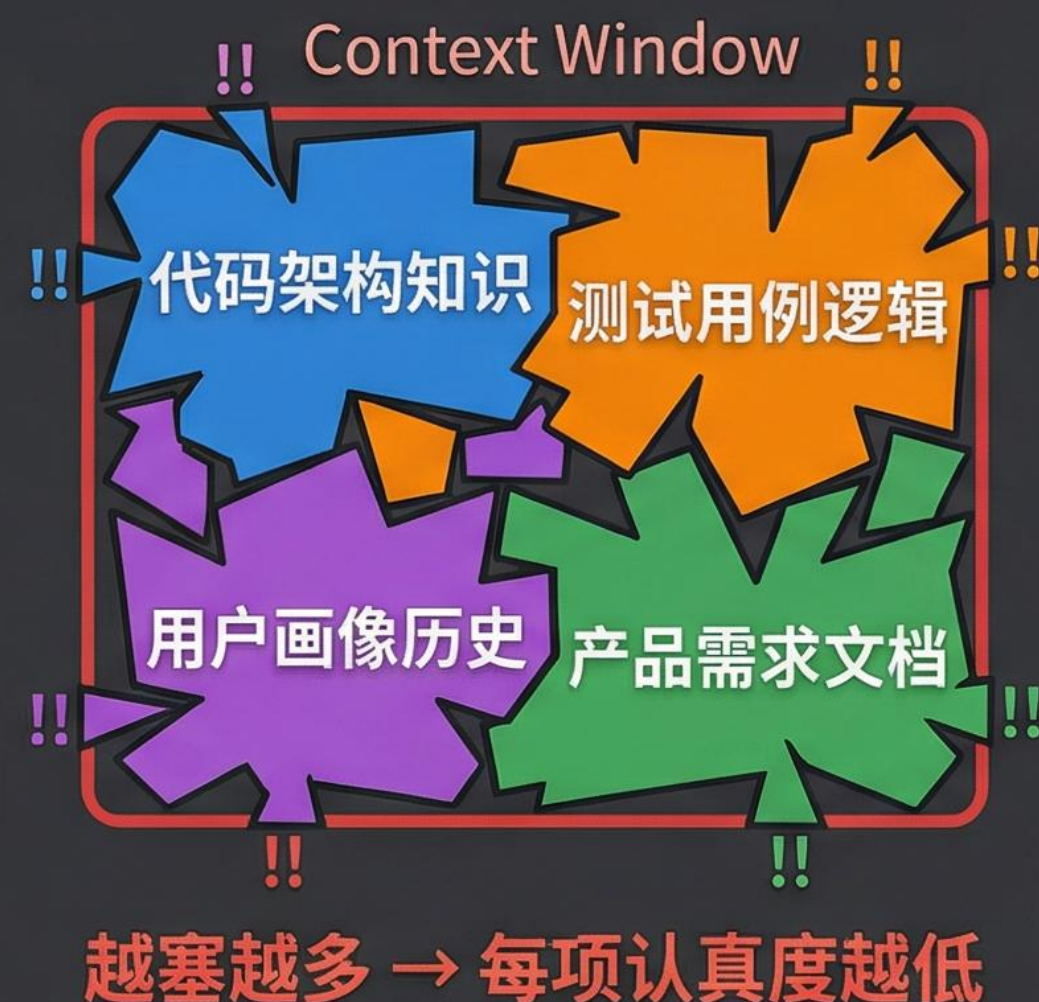
角色体系 = 给每个数字员工颁发职业执照

错误叠加是乘法，不是加法

边界清楚了，系统才能被推理、被测试、被维护

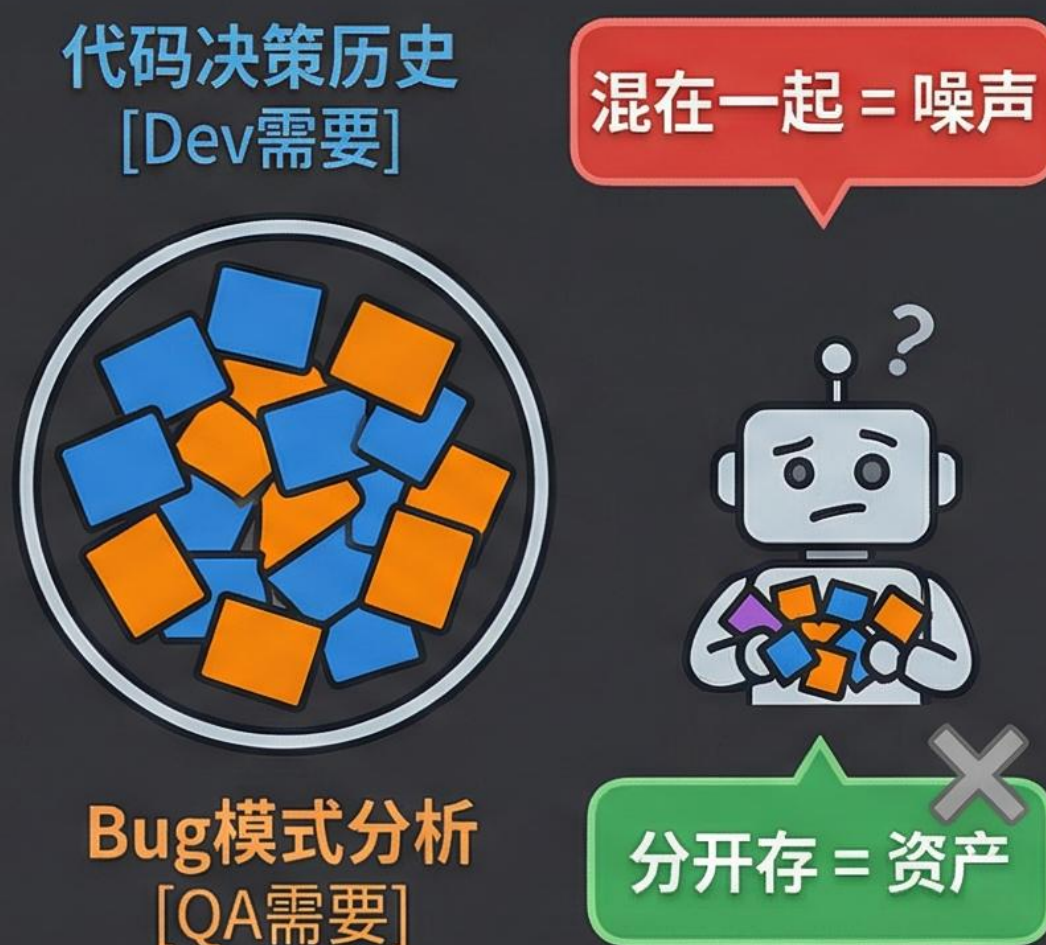
通才模式的三个崩溃场景

崩溃一：Context 装不好通才



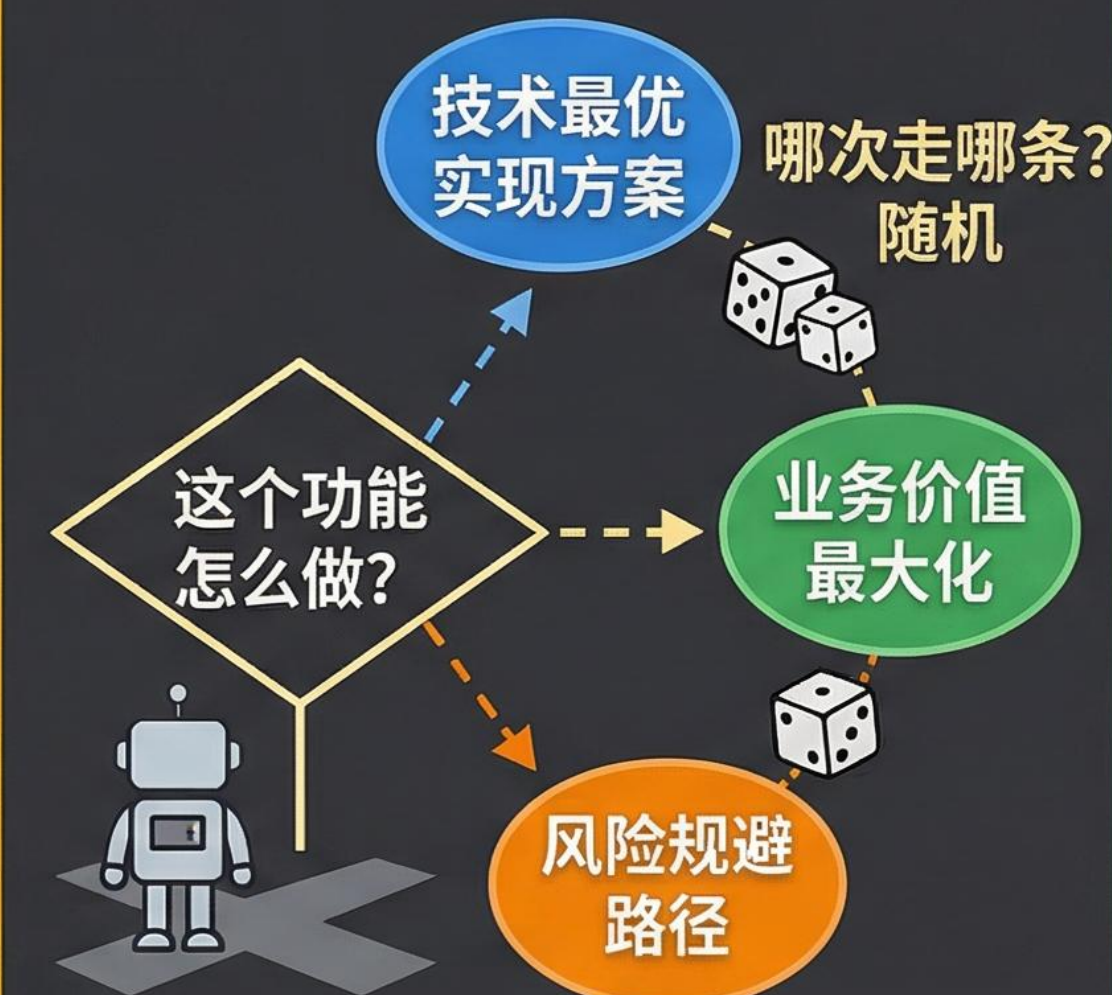
方向分裂，积累停摆

崩溃二：记忆混成噪声



谁的经验也积累不深

崩溃三：判断方向随机

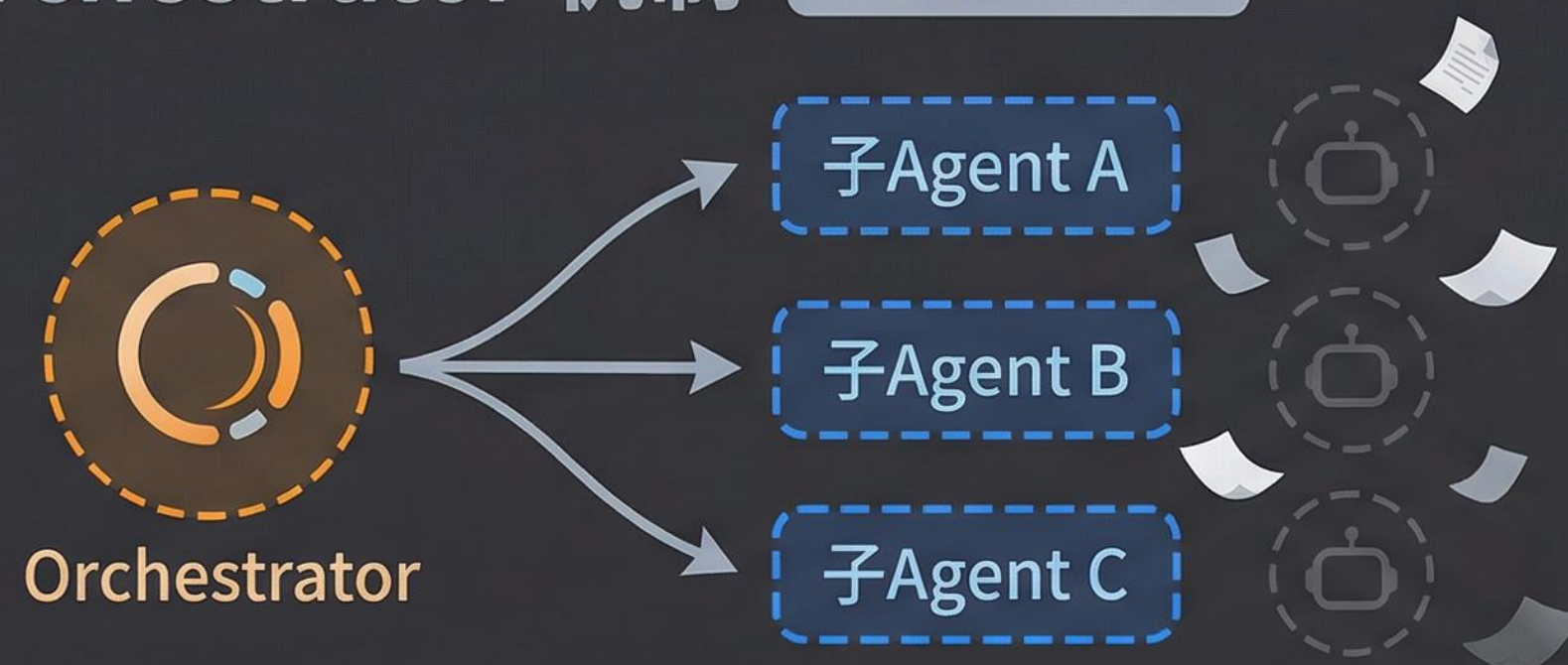


生产环境里，随机判断=不可靠

三个崩溃合在一起，解释了为什么'多几个 Agent'反而更难用

行业选择：中心协调 + Manager ≠ Orchestrator

Orchestrator 机制 任务完成即散



features

- ▶ 临时创建，任务结束即销毁
- ▶ 无 soul，无记忆，无身份
- ▶ 上下文在任务结束后清空

用完即抛的
调度机制

Manager 角色 持久团队成员



features

- ▶ 团队里有名有姓的协调者
- ▶ 跨 session 积累分工经验
- ▶ 知道全局：任务优先级/角色状态

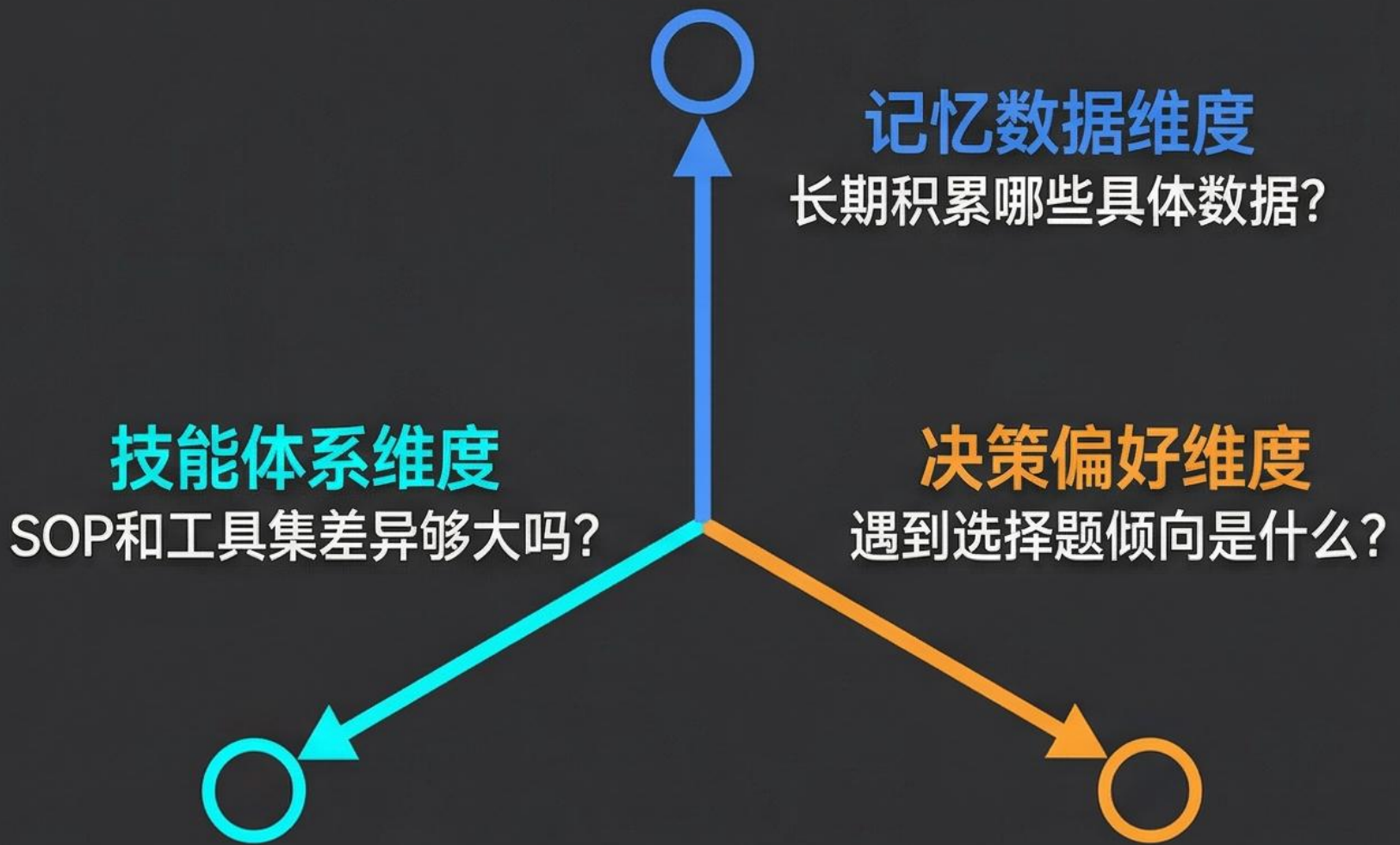
持久化数字
员工，有身份

↑
↓ 本质升级

一个是调度机制，一个是团队成员——混淆这两个，团队就建不起来

三维隔离判断法：除了Manager，谁应该独立存在

3D 维度的方法



- 三个维度都有显著差异 → 值得独立建角色 ✓
- 只有一个维度不同 → 扩展现有角色 Skills ✗ (不新建)

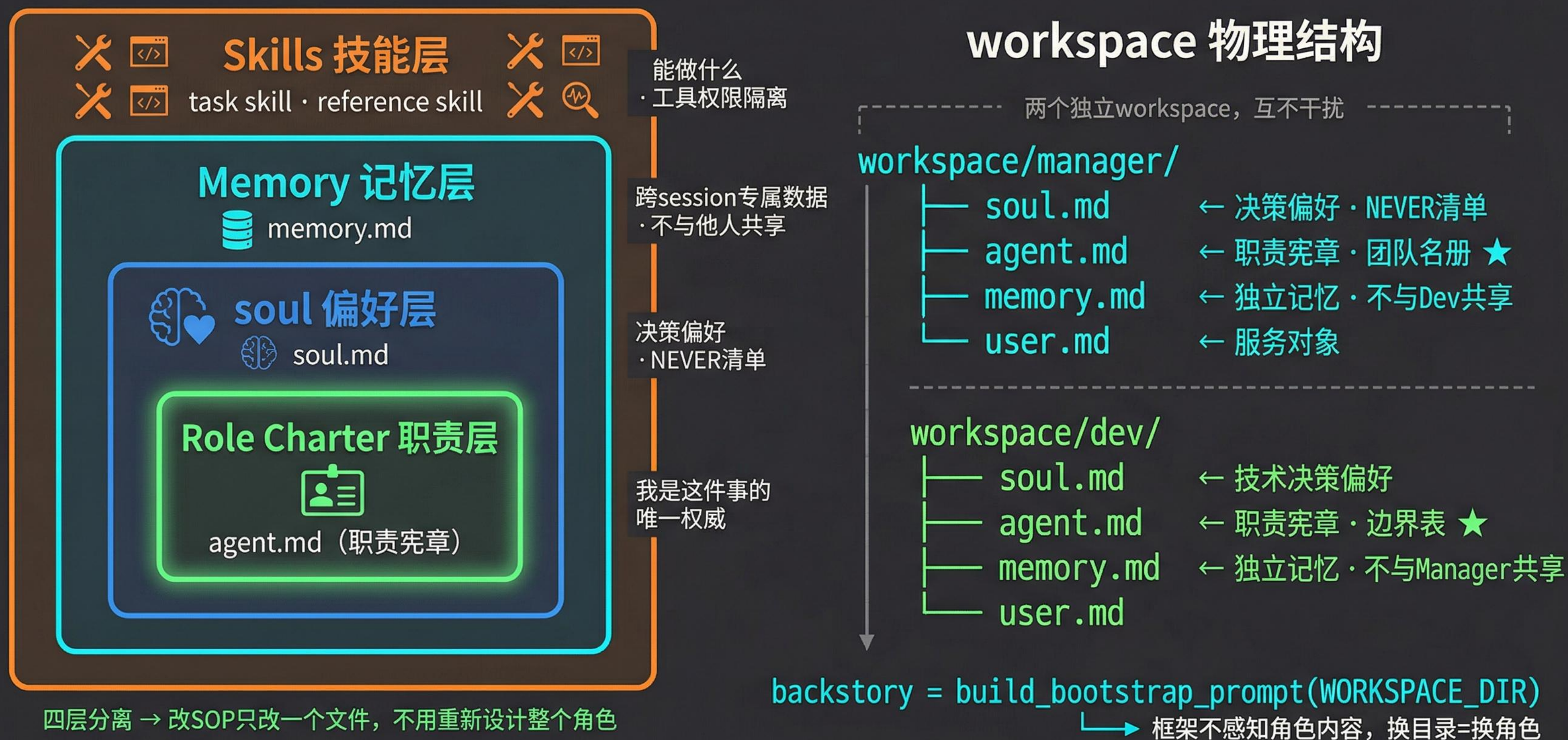
Dev vs QA 实例对比

维度	Dev	QA
记忆数据	代码库架构历史/ 技术决策记录	测试用例库/ 历史Bug模式
技能体系	代码实现SOP/ 代码执行工具	测试设计SOP/ 测试执行工具
决策偏好	技术实现最优	风险规避优先

反例：会用Figma的Dev
记忆数据（代码库）= 和Dev相同 ✗，
决策偏好（技术最优）= 和Dev相同 ✗，只是多了一个工具
→ 不建新角色，扩展Dev的Skills

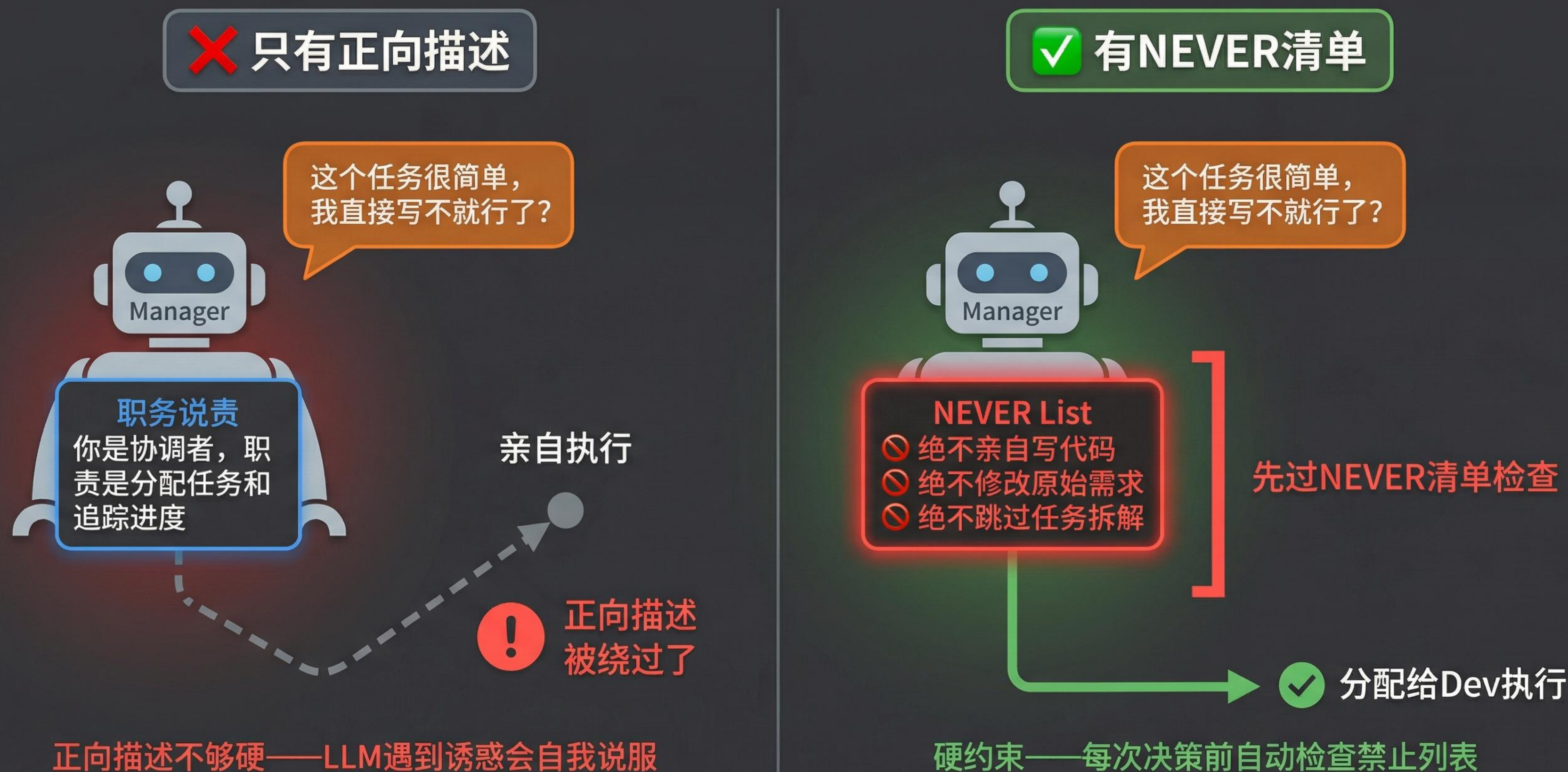
新增角色有代价——协调成本会吃掉并发收益，默认不新建

代码实战：四层框架 + workspace 物理结构



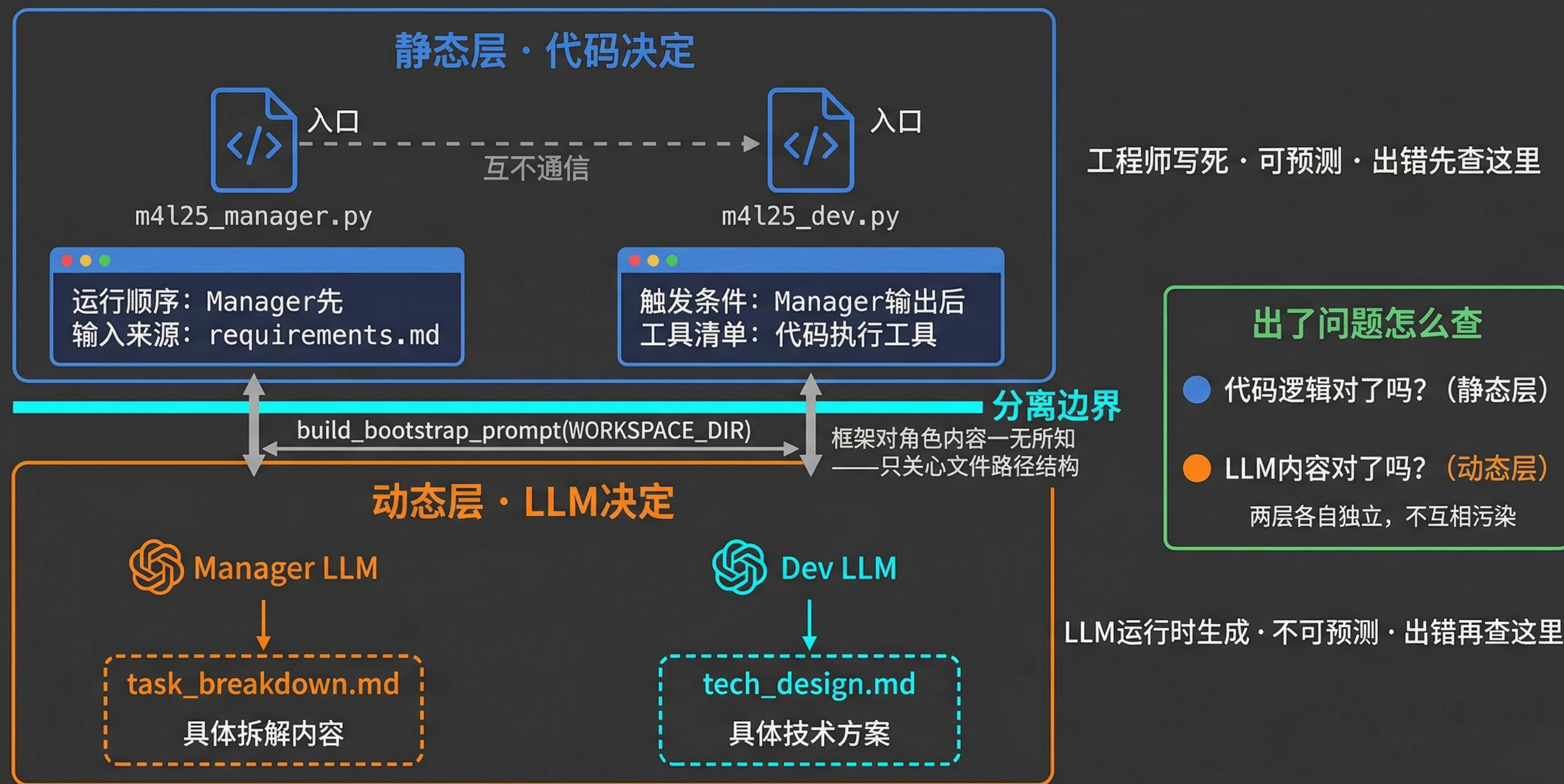
框架对角色内容一无所知——这就是整个团队可扩展性的根基

NEVER 清单：边界约束比职责描述更难被突破



‘你不能做X’的约束力，强于‘你的职责是Y’的描述

代码决定结构，LLM 决定内容——两层各自独立排查



把最难调试的 LLM 行为，限制在最小的单元——这是整个系统可维护性的根基

最佳实践与反模式

反模式

Manager 也亲自执行业务任务：

没有 NEVER 清单的 Manager，失去全局视野，整个团队失去中枢，执行效率更低。

职责（Role Charter）不清：

Dev 和 QA 都认为"单元测试"归自己；

或者反过来，两个角色互相推诿都认为"这是对方的事"

Agent过度分化

协调成本大幅上升，影响性能

最佳实践

NEVER 清单+工具skills限制共同划边界：

双重限制保证职责清晰

Memory 目录按 agent_id 严格隔离：

命名规范 workspace/{agent_id}/。物理路径隔离是"架构保证"，不是"LLM 自律"。

新增角色前先用三维判断法，默认不新建：

团队越小越好维护——协调成本会吃掉并发收益

课程总结

理解了数字员工的角色分工

学习了中心协调者模式的团队构建

清晰认识数字员工独立的4要素：职责，soul，记忆，技能

下节提示：如何让数字员工协作起来：任务链与信息传递